Design e Arquitectura de Software

2022/2023

Lab. Classes - 2

GoF - Behavioral Patterns

1. Implement the basic **Template Method** example illustrated in slides nr. 9-13.
   An example of expected output is:

```
Creating new spreadsheet application.
SpreadsheetApplication: Attempting to open file "file 1".
SpreadsheetApplication: Can open "file 1".
SpreadsheetApplication: Creating spreadsheet document object
Application: Adding new document to document list
Application: Notifying about to open document
SpreadsheetApplication: Doc about to be opened, how exciting!
SpreadsheetDocument: Opening "file1" spreadsheet file
SpreadsheetDocument: Reading contents of "file1" spreadsheet
file
```

2. Implement the **Observer** pattern, as illustrated in slide nr. 18, <u>without</u> resorting to support from the API classes (Observer/Observed, PropertyChangeSupport/Listener, etc).
   An example of expected output is:

```
Working with the company: Abc Ltd.
ABC Ltd. registers Roy
ABC Ltd. registers Kevin
ABC Ltd. registers Bose
ABC Ltd.'s current stock price is $5.
Roy has received an alert from ABC Ltd. The current stock price
is:$5
Kevin has received an alert from ABC Ltd. The current stock
price is:$5
Bose is notified from ABC Ltd.Its current stock price is:$5
-----
ABC Ltd. is removing Kevin from the observer list now.
ABC Ltd. unregisters Kevin
ABC Ltd.'s new stock price is $50.
Roy has received an alert from ABC Ltd. The current stock price
is:$50
Bose is notified from ABC Ltd.Its current stock price is:$50
-----
Kevin registers again to get notifications from ABC Ltd.
ABC Ltd. registers Kevin
ABC Ltd.'s new stock price is $100.
Roy has received an alert from ABC Ltd. The current stock price
is:$100
Bose is notified from ABC Ltd.Its current stock price is:$100
Kevin has received an alert from ABC Ltd. The current stock
price is:$100
-----
Working with another company: XYZ Co.
XYZ Co. registers Roy
```

```
XYZ Co. registers Jacklin
XYZ Co.'s new stock price is $500.
Roy has received an alert from XYZ Co. The current stock price
is:$500
Jacklin is notified from XYZ Co. Its current stock price is:$500
```

3. Implement the previous example using API support:
   - Resorting to the deprecated `Observer/Observable` classes.
   - Resorting to the java beans package: `PropertyChangeListener`, `PropertyChangeSupport, etc`
     (if you are unfamiliar with it, see example at https://gist.github.com/mtorchiano/e69ac7e309fee81bd17f4f0740b9ffa9)

4. Implement the **chain of command** example illustrated in slide nr. 23. Consider that:
   - The application includes a list of Widgets.
   - The application should always provide a default help request.
   - The widgets form a composite pattern.

   An example of expected output is:

```
Creating new Application providing default help
Creating new PrintDialog. It does not handle help
Creating new DrawDialog. It handles help.
Creating new Button1. It does not handle help.
Creating new Button2. It handles help.
Creating new Button3. It does not handle help.
Adding Button1 to DrawDialog
Adding Button2 to DrawDialog
Adding Button3 to PrintDialog
Button1:: request for help, pass it along the chain.
DrawDialog:: Help request received, will handle
DrawDialog:: Showing help for draw dialog
Button2:: Help request received, will handle
Button2:: Showing help for button 2
Button3:: request for help, pass it along the chain.
PrintDialog:: request for help, pass it along the chain.
Application:: Help request received, will handle
Application:: Showing help for application
```

5. Implement, using Java **Iterators**, a method `checkAbove` that receives any data structure providing an iterator and identifies the percentage of integer numbers above a threshold given by a parameter.

   An example of use would be:

```
Integer a[]=new Integer[]{0,1,2,3,4,5,6,7,8,9}
List<Integer> lst=Arrays.asList(a);
Set<Integer> set=new TreeSet<>(lst);
System.out.println("% in list is "+checkAbove(lst,5));
System.out.println("% in set is "+checkAbove(set,5));
```

6. Implement the **Command** pattern, considering the following (refer to slide 31 for terminology):
   - The receiver object is a two-dimensional point with x and y coordinates.
   - The possible operations are moving north, south, east or west.
   - The command objects and interface should support the undoing of operations.
   - The client code creates and sends commands to the invoker to execute.
   - The invoker object should maintain an undolist, and a redo list, storing commands in chronological order allowing for undoing/redoing of operations.
   - Use a macro command to represent moving 10x in the NW direction.

   Here is an example of expected output:

```
Client: Creating new 2d position@ 0,0
Client: Creating new command (Move North)
Client: Sending command to invoker
Invoker: Receiving new command
Invoker: Storing new command in undo list and clearing redo list (undo
list: 1 commands, redo list: 0 commands)
Invoker: executing command
Command: Executing. Calling SetY() on 2d position
Point2D: Moving to 0,1.
Client: Creating new command (Move East)
Client: Sending command to invoker
Invoker: Receiving new command
Invoker: Storing new command in undo list and clearing redo list (undo
list: 2 commands, redo list: 0)
Invoker: executing command
Command: Executing. Calling SetX() on 2d position
Point2D: Moving to 1,1.
Client: Creating new command (Move North)
Client: Sending command to invoker
Invoker: Receiving new command
Invoker: Storing new command in undo list and clearing redo list (undo
list: 3 commands, redo list: 0)
Invoker: executing command
Command: Executing. Calling SetY() on 2d position
Point2D: Moving to 1,2.
Client: Sending request to undo to invoker
Invoker: Received undo request
Invoker: Undoing last command in undo list
Command: Undoing. Calling SetY() on 2d position
Point2D: Moving to 1,1.
Invoker: Removing last command from undo list and adding to redolist
(undo list: 2 commands, redo list: 1)
Client: Creating new command (Move East)
Client: Sending command to invoker
Invoker: Receiving new command
Invoker: Storing new command in undo list and clearing redo list (undo
list: 3 commands, redo list: 0)
Invoker: executing command
Command: Executing. Calling SetX() on 2d position
Point2D: Moving to 2,1.
Client: Sending request to redo to invoker
Invoker: Received redo request
Invoker: Not possible, no previously undone command found
```